

컴퓨터 구조1_과제

리눅스 개발 환경 만들기

→ 기존에 WLS2와 우분투를 설치해 리눅스 환경을 이용하기 있었기 때문에, 구름은 사용하지 않고 이것을 개발 환경으로 사용하였다.

Sizeof 연산 타이핑 해보기

[결과]

```
changwoo@DESKTOP-H6FJ1FE:~/WHS$ gcc -o sizeof sizeof.c
changwoo@DESKTOP-H6FJ1FE:~/WHS$ ls
sizeof  sizeof.c
changwoo@DESKTOP-H6FJ1FE:~/WHS$ ./sizeof
Size of char: 1 bytes
Size of short: 2 bytes
Size of int: 4 bytes
Size of long: 8 bytes
Size of long long: 8 bytes
Size of float: 4 bytes
Size of double: 8 bytes
Size of long double: 16 bytes
Size of pointer: 8 bytes
```

[몰랐던 것]

1. %zu?

→ C언어에서 size_t에 대응하는 형식지정자로, printf 함수에 sizeof를 직접 넣을 때 사용한다.

2. size_t?

→ 해당 시스템에서 어떤 객체나 값이 포함할 수 있는 최대 크기의 데이터를 표현하는 타입이다.

→ 대표적으로 sizeof, malloc, strlen 등 크기를 다루는 함수에서 사용한다.

3. 포인터 크기?

→ 32bit 환경, 64bit 환경에 따라 4byte, 8byte로 변한다.

오버플로 예제를 언더 플로로 바꿔서 해보기

[코드]

```
#include <stdio.h>
#include <limits.h>

int main(){
    char value = CHAR_MIN;

    printf("Original value: %d\n", value);

    value = value - 1;

    printf("Value after sub 1: %d\n", value);

    return 0;
}
```

[결과]

- 언더플로 결과

```
changwoo@DESKTOP-H6FJ1FE:~/WHS$ ./underflow
Original value: -128
Value after sub 1: 127
```

- gdb main

```

changwoo@DESKTOP-H6FJ1FE:~/WHS$ gdb underflow
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Poetry could not find a pyproject.toml file in /home/changwoo/WHS or its par
ents
pwndbg: loaded 160 pwndbg commands and 47 shell commands. Type pwndbg [--she
ll | --all] [filter] for a list.
pwndbg: created $rebase, $base, $ida GDB functions (can be used with print/b
reak)
Reading symbols from underflow...
----- tip of the day (disable with set show-tips off) -----
break-if-taken and break-if-not-taken commands sets breakpoints after a give
n jump instruction was taken or not

```

- b main

```

pwndbg> b main
Breakpoint 1 at 0x1155: file underflow.c, line 5.

```

- r

```

pwndbg> r
Starting program: /home/changwoo/WHS/underflow

```

- n

```

pwndbg> n
7          printf("Original value: %d\n", value);

```

- p value & p/t value
 - 언더플로 전의 value 값

```
pwndbg> p value
$2 = -128 '\200'
pwndbg> p/t value
$3 = 10000000
```

- 언더플로 이후의 value 값

```
pwndbg> p value
$4 = 127 '\177'
pwndbg> p/t value
$5 = 1111111
```

[몰랐던 것]

1. -g?(왜 디버깅 심볼을 넣어주나?)

→ 디버깅 정보가 포함됨 → gdb 디버거로 쉽게 분석 가능

2. limits.h

→ CHAR_MIN을 사용하기 위한 헤더

비트 연산 프로그램 바꿔보기

[코드]

```
#include <stdio.h>

int is_bit_set(unsigned char value, int position){
    return (value & (1<<position)) != 0;
}

unsigned char set_bit(unsigned char value, int position){
```

```

        return value | (1<<position);
    }

    unsigned char clear_bit(unsigned char value, int position){
        return value & ~(1<<position);
    }

    int main(){
        unsigned char value = 0b00001000;
        int position = 0;

        if(is_bit_set(value,3)){
            printf("3rd bit is set!\n");
        }else{
            printf("3rd bit is not set!\n");
        }

        value = set_bit(value, 2);
        printf("Value after setting 2nd bit: %d\n", value);

        while(position != 9){
            printf("Input position (only int, if you want to exit? then input 9) ");
            scanf(" %d", &position);

            value = clear_bit(value, position);

            if (position == 9){
                printf("Exit\n");
                break;
            }

            printf("Value after clearing %d bit: %d\n", position, value);
        }

        return 0;
    }

```

[결과]

```
changwoo@DESKTOP-H6FJ1FE:~/WHS$ ./clear_bit
3rd bit is set!
Value after setting 2nd bit: 12
Input position (only int, if you want to exit? then input 9) 1
Value after clearing 1 bit: 12
Input position (only int, if you want to exit? then input 9) 2
Value after clearing 2 bit: 8
Input position (only int, if you want to exit? then input 9) 3
Value after clearing 3 bit: 0
Input position (only int, if you want to exit? then input 9) 4
Value after clearing 4 bit: 0
Input position (only int, if you want to exit? then input 9) 9
Exit
```

[몰랐던 것]

1. 비트 연산자들

C언어가 기계어가 되는 과정 직접 해보기

- gcc -E main.c -o main.i 실행

```
changwoo@DESKTOP-H6FJ1FE:~/WHS$ vi main.c
changwoo@DESKTOP-H6FJ1FE:~/WHS$ gcc -E main.c -o main.i
changwoo@DESKTOP-H6FJ1FE:~/WHS$ ls
bit bit.c clear_bit main.c main.i sizeof sizeof.c test underflow.c
```

- .i 파일 확인하기

```
# 0 "main.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "main.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 27 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 392 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 393 "/usr/include/features.h" 2 3 4
# 486 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 559 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 560 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 561 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 487 "/usr/include/features.h" 2 3 4
# 510 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 511 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 28 "/usr/include/stdio.h" 2 3 4
```

- gcc -S main.i -i main.s 실행

```
changwoo@DESKTOP-H6FJ1FE:~/WHS$ gcc -S main.i main.s
changwoo@DESKTOP-H6FJ1FE:~/WHS$ ls
bit      clear_bit  main.i      sizeof      test
bit.c    main.c      main.s      sizeof.c    underflow.c
```

- .s 파일 확인하기

```

        .file      "main.c"
        .text
        .section   .rodata
.LC0:
        .string   "Hello, World!"
        .text
        .globl    main
        .type     main, @function
main:
.LFB0:
        .cfi_startproc
        endbr64
        pushq    %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq     %rsp, %rbp
        .cfi_def_cfa_register 6
        leaq     .LC0(%rip), %rax
        movq     %rax, %rdi
        call     puts@PLT
        movl     $0, %eax
        popq     %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size     main, .-main
        .ident    "GCC: (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0"
        .section  .note.GNU-stack,"",@progbits
        .section  .note.gnu.property,"a"
        .align    8
        .long     1f - 0f
        .long     4f - 1f
        .long     5
0:
        .string   "GNU"
1:
        .align    8
        .long     0xc0000002
        .long     3f - 2f

```

- gcc -c main.s main.o 실행

```

changwoo@DESKTOP-H6FJ1FE:~/WHS$ gcc -c main.s -o main.o
changwoo@DESKTOP-H6FJ1FE:~/WHS$ ls
bit      clear_bit  main.i    main.s    sizeof.c  underflow.c
bit.c    main.c      main.o    sizeof    test

```


- readelf -a main.o

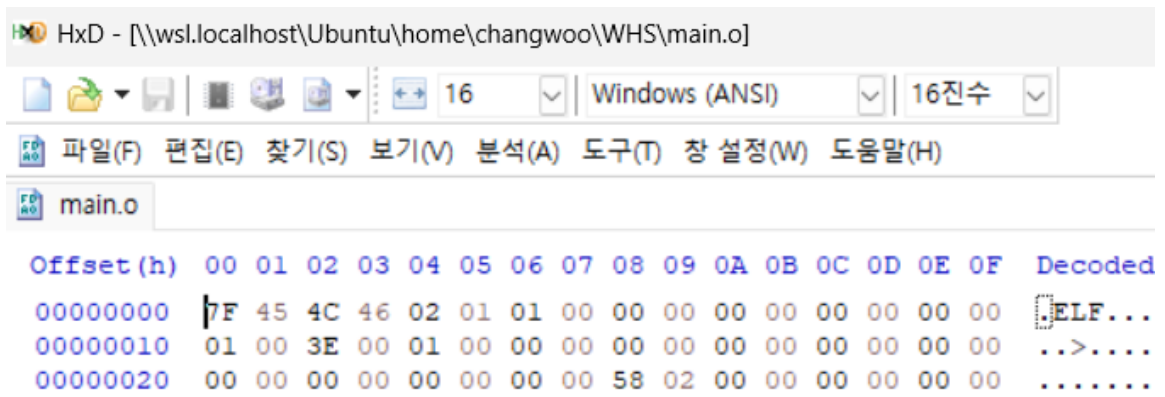
```

changwoo@DESKTOP-H6FJ1FE:~/WHS$ readelf -a main.o
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  REL (Relocatable file)
  Machine:                                Advanced Micro Devices X86-64
  Version:                                0x1
  Entry point address:                    0x0
  Start of program headers:               0 (bytes into file)
  Start of section headers:              600 (bytes into file)
  Flags:                                  0x0
  Size of this header:                    64 (bytes)
  Size of program headers:                0 (bytes)
  Number of program headers:              0
  Size of section headers:                64 (bytes)
  Number of section headers:              14
  Section header string table index:      13

Section Headers:
 [Nr] Name              Type              Address              Offset
      Size              EntSize          Flags  Link  Info  Align
 [ 0]                      NULL              0000000000000000    00000000
      0000000000000000  0000000000000000                0      0      0
 [ 1] .text               PROGBITS          0000000000000000    00000040
      000000000000001e  0000000000000000  AX      0      0      1
 [ 2] .rela.text          RELA              0000000000000000    00000198
      0000000000000030  0000000000000018  I       11     1      8
 [ 3] .data               PROGBITS          0000000000000000    0000005e
      0000000000000000  0000000000000000  WA      0      0      1
 [ 4] .bss                NOBITS            0000000000000000    0000005e
      0000000000000000  0000000000000000  WA      0      0      1

```

- HxD로 ELF헤더 확인



- objdump -d main.o로 확인

```
changwoo@DESKTOP-H6FJ1FE:~/WHS$ objdump -d main.o

main.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
 0:  f3 0f 1e fa          endbr64
 4:  55                   push    %rbp
 5:  48 89 e5             mov     %rsp,%rbp
 8:  48 8d 05 00 00 00 00 lea     0x0(%rip),%rax        # f <main+0xf>
 f:  48 89 c7             mov     %rax,%rdi
12:  e8 00 00 00 00       call   17 <main+0x17>
17:  b8 00 00 00 00       mov     $0x0,%eax
1c:  5d                   pop     %rbp
1d:  c3                   ret
```

- gcc main.o -o main 및 실행

```
changwoo@DESKTOP-H6FJ1FE:~/WHS$ gcc main.o -o main
changwoo@DESKTOP-H6FJ1FE:~/WHS$ ls
bit bit.c clear_bit main main.c main.i main.o
changwoo@DESKTOP-H6FJ1FE:~/WHS$ ./main
Hello, World!
```

- 실행파일과 오브젝트 파일의 차이

```
main:      ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f14bf2e15cab179d82a09a2de5bf15da6e5b75c, for GNU/Linux 3.2.0, not stripped

main.o:    ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
```

